Mini Project Report

on

WanderLust

Bachelor of Technology

In

Computer Science and Engineering

By

1. Naman Sharma

Roll No.: 2406950100108

2. Prerak

Roll No.: 2406950100127

Under the Supervision of

Tanu Verma

Assistant Professor

Department of Computer Science and Engineering



SHRI RAM GROUP OF COLLEGES
MUZAFFARNAGAR

AFFILIATED TO

Dr A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY,
UTTAR PRADESH, LUCKNOW

(2025-26)

DECLARATION

We, Naman Sharma (Roll No. 2406950100108) and Prerak (Roll No. 2406950100127), students of the Department of Computer Science and Engineering, in the third year of the Bachelor of Technology (B.Tech) program at Shri Ram Group of Colleges, hereby declare that the mini project titled "WanderLust" is the result of our work and research. The work presented in this report has been carried out under the supervision of Tanu Verma, Assistant Professor, Department of Computer Science and Engineering, Shri Ram Group of Colleges.

This report has not been submitted to any other university or institution for the award of any degree or diploma. The technologies and documentation used in developing the application are acknowledged in the references.

Date: _____

Place: _____

Signature: _____
Naman Sharma
Roll No. 2406950100108
B.Tech CSE, Third Year

Signature: _____
Prerak
Roll No. 2406950100127
B.Tech CSE, Third Year

Shri Ram Group of Colleges

Academic Session: 2025-26

CERTIFICATE

This is to certify that the mini project titled "WanderLust" has been carried out by Naman Sharma (Roll No. 2406950100108) and Prerak (Roll No. 2406950100127), students of the Department of Computer Science and Engineering, in the third year of the Bachelor of Technology (B.Tech) program at Shri Ram Group of Colleges. The project report is prepared as part of the academic requirements for the mini project course during the session 2025-26.

The work has been conducted under the guidance and supervision of Tanu Verma, Assistant Professor, Department of Computer Science and Engineering, Shri Ram Group of Colleges.

The project demonstrates the application of web development concepts to travel discovery and accommodation listing management. The implementation, verification evidence and current limitations are described in this report.

Students' signatures

_____

Naman Sharma
2406950100108

_____

Prerak
2406950100127
B.Tech CSE, Third Year

Supervisor's signature

_____

Tanu Verma
Assistant Professor
CSE Department
Shri Ram Group of Colleges

ACKNOWLEDGEMENTS

We are pleased to present the report of our B.Tech Mini Project "WanderLust", undertaken during our third year at Shri Ram Group of Colleges, Muzaffarnagar, in the academic session 2025-26.

We express our sincere gratitude to Tanu Verma, Assistant Professor, Department of Computer Science and Engineering, for her guidance and support throughout this project. Her suggestions helped us organize the work and improve the application and its documentation.

We thank the faculty members of the Department of Computer Science and Engineering for their assistance and cooperation. We also acknowledge Shri Ram Group of Colleges for the academic environment in which we could apply our knowledge of web application development.

We are grateful to our parents and friends for their encouragement. We acknowledge the official documentation and open-source technologies that supported the implementation and verification of WanderLust.

Naman Sharma - Roll No. 2406950100108

Prerak - Roll No. 2406950100127

B.Tech CSE, Third Year

Shri Ram Group of Colleges

ABSTRACT

WanderLust is a travel accommodation listing application designed to help visitors discover stays and help registered users manage property information. It brings titles, photographs, descriptions, destinations, nightly prices and reviews into a responsive web interface. The application uses Node.js and Express.js for request handling, MongoDB with Mongoose for storage, and EJS with Bootstrap for server-rendered presentation.

The implemented discovery features include literal search across title, location and country, inclusive price filters, price sorting and server-side pagination. Users can save stays in the current browser, open location-area map links and explore related listings. Review summaries use actual review documents. Existing authentication, listing management and review workflows remain part of the application.

Verification evidence includes eighteen unit tests, valid and invalid HTTP query cases, pagination checks, responsive layout checks and automated accessibility scans. Thirty-two fictional demonstration listings provide repeated destinations and varied prices for search verification. Their import uses stable identifiers and add-only behavior. Successful uploads, complete authenticated mutation workflows and live deployment behavior were not independently verified in the cloud environment. Booking, payments and account-synchronized saved stays remain future work.

Keywords: travel discovery, accommodation listings, Express.js, MongoDB, EJS, search, responsive interface.

TABLE OF CONTENTS

Declaration	i
Certificate	ii
Acknowledgements	iii
Abstract	iv
List Of Figures And Tables	vi
Chapter 1: Introduction	1
Chapter 2: Problem Statement	2
Chapter 3 Implementation and database design	3
3.2 Discovery and user features	4
3.4 User interface screenshots	5
3.5 Testing and demo data	6
Chapter 4: Conclusion And Future Work	7
References	8

LIST OF FIGURES AND TABLES

LIST OF FIGURES

Figure 1 Request and storage architecture 3
Figure 2 User Listing and Review relationships 3
Figure 3 Listings interface in local development 5
Figure 4 Manali search with price filters 5

LIST OF TABLES

Table 2.1 Technologies and responsibilities 2
Table 3.1 Supported discovery parameters 4
Table 3.2 Verification results 6

Chapter 1

INTRODUCTION

1.1 PROJECT OVERVIEW

WanderLust is a web application for discovering and managing travel accommodation listings. It brings property titles, photographs, descriptions, destinations, nightly prices and reviews into one responsive interface. Visitors can compare stays, while registered users can contribute listings and reviews.

The project applies full-stack development through the existing Node.js, Express.js, MongoDB, Mongoose, EJS and Bootstrap architecture. EJS runs on the server and produces the HTML returned to the browser. Database queries provide discovery results rather than browser-only filtering.

1.2 OBJECTIVES

• Help visitors find stays by title, location, country and nightly budget.

• Provide predictable sorting and manageable result pages.

• Retain authentication, listing ownership and genuine review workflows.

• Improve discovery through device-local saves, related stays and map links.

1.3 SCOPE

The implemented scope covers travel discovery, listing management and community reviews. Visitors can browse without signing in; contribution actions use the existing authentication controls. A listing owner can access its management actions.

Saved stays remain in the current browser. Booking, payment processing, availability calendars and account-synchronized saves are future extensions. No fictional review scores or booking functionality are presented as completed features.

Chapter 2

PROBLEM STATEMENT

2.1 PROBLEM AND REQUIREMENTS

Travelers need relevant destination and budget results without inspecting every property. WanderLust connects discovery controls to MongoDB, preserves selected values and presents clear empty states. Protected contributions retain the existing authentication and ownership relationships.

2.2 TECHNOLOGIES USED

Table 2.1 Technologies and responsibilities

Technology	Responsibility
Node.js / Express	Server runtime, routes, middleware and responses.
MongoDB / Mongoose	Document storage, models and database queries.
EJS / Bootstrap	Server-rendered pages and responsive controls.
Passport / sessions	Authentication and persistent login sessions.
Joi	Discovery, listing and review input validation.
Multer / Cloudinary	Configured image-upload and storage integration.
localStorage	Saved listing IDs in the current browser.

2.3 SETUP AND FEASIBILITY

Install using npm ci --legacy-peer-deps and start with node app.js. The application normally serves port 8080. Keep database and upload credentials private. The destructive initializer is excluded from normal setup.

Chapter 3

IMPLEMENTATION AND RESULTS

3.1 ARCHITECTURE AND DATABASE DESIGN

Routes and middleware process requests; controllers query Mongoose models; EJS renders responses. Saved preferences stay in browser storage, and uploads use the configured Cloudinary integration.

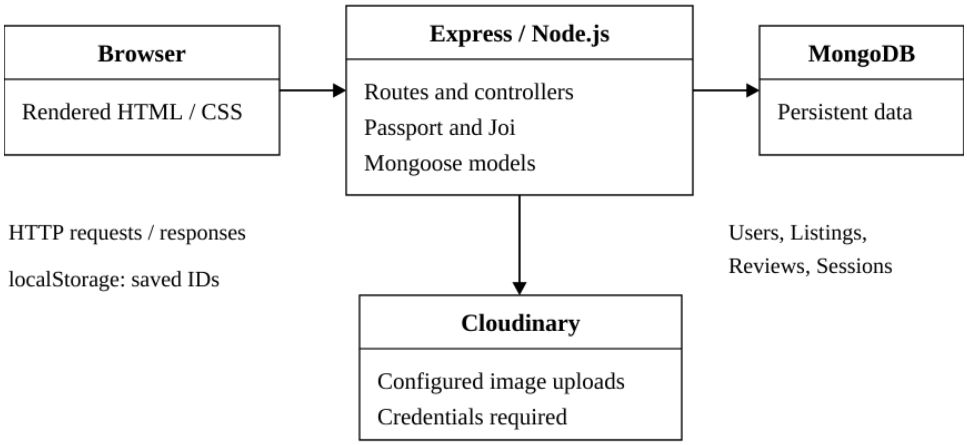


Figure 1 Request and storage architecture

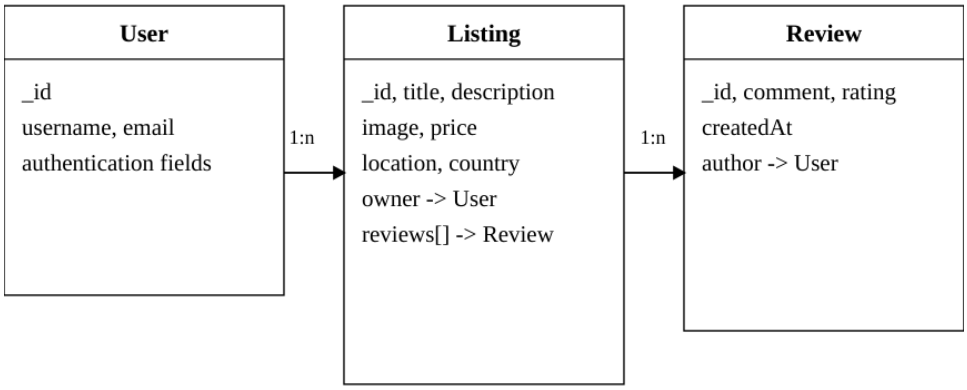


Figure 2 User Listing and Review relationships

Listing fields include title, description, image, price, location and country, with owner and review references. Review records reference their author. The current features require no new model fields.

DISCOVERY AND USER FEATURES

3.2 SEARCH FILTERS AND PAGINATION

Table 3.1 Supported discovery parameters

Parameter	Behavior
q	Literal search over title, location and country.
minPrice / maxPrice	Inclusive minimum and maximum nightly prices.
sort	default, price_asc or price_desc only.
page	Validated positive page number; 12 stays per page.

The server rejects malformed scalar values, reversed ranges and unsupported sorts. It escapes regex characters and builds explicit MongoDB conditions. Search text is limited to 100 characters; prices must be non-negative. Pagination retains filters and uses stable sorting.

3.3 OTHER IMPLEMENTED FEATURES

- Saved stays: save up to 100 IDs on this device; reload persistence and retry behavior are supported, without account synchronization.
- Reviews: averages use genuine valid ratings; empty records show "No reviews yet".
- Related stays and maps: show up to four other stays and open an encoded location-area search, without claiming an exact address.
- Forms and images: validation feedback, duplicate-submission prevention, selected-photo preview and truthful failed-image fallbacks improve usability.

Authentication, listing CRUD and review routes remain within the original architecture. Actual successful mutation workflows require additional integration verification.

SCREENSHOTS OF USER INTERFACE

Local development captures show the listings and filtered-results interfaces. Labelled image fallbacks reflect restricted external photo access.

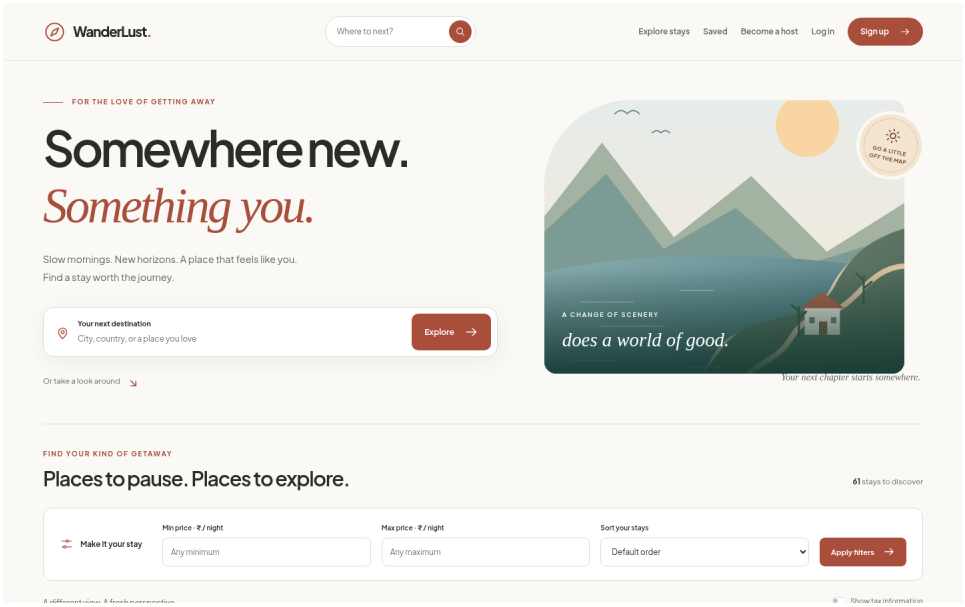


Figure 3 Listings interface in local development

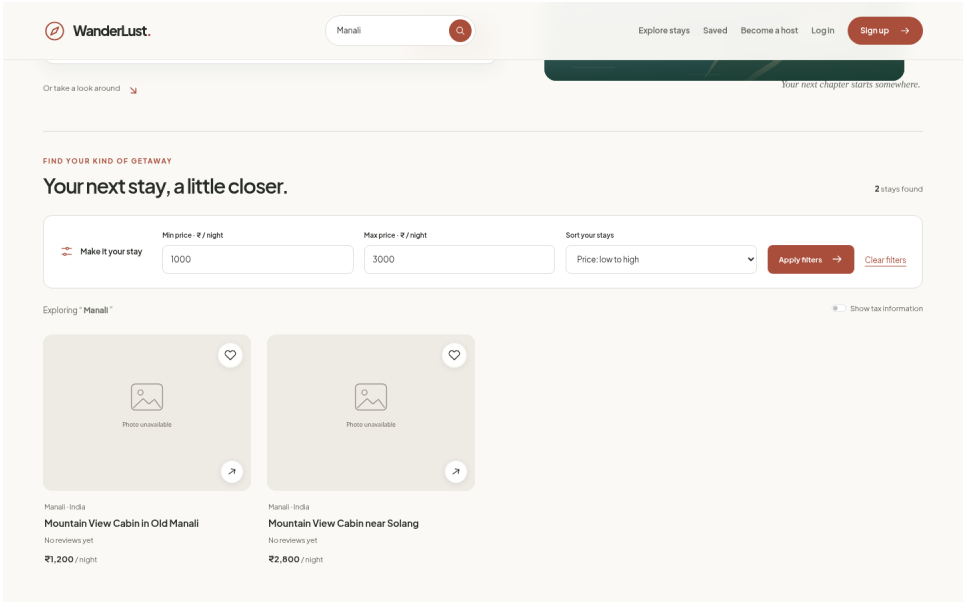


Figure 4 Manali search with price filters

TESTING AND DEMONSTRATION DATA

3.5 RECORDED LOCAL VERIFICATION

Table 3.2 Verification results

Check group	Recorded result
Logic and seed safety	18 unit tests passed.
HTTP queries	17 valid and 10 invalid cases passed.
Pagination	15 pages checked without gaps or duplicates.
Responsive layouts	45 checks at 320, 375, 768, 1024 and 1440 pixels.
Accessibility	20 scans; zero automated violations detected.

3.6 DATASET AND SEARCH EXAMPLE

Thirty-two fictional demo stays cover twenty destinations and INR 800 to INR 25,000 per night. Manali, Goa, Jaipur, Munnar and major Indian cities are represented alongside four international destinations. Stable IDs allow an add-only, repeatable import without overwriting existing records.

Manali with INR 1,000 to INR 3,000 matches two demo stays priced INR 1,200 and INR 2,800. Other records may add matches. Full and partial titles, locations, empty queries, no matches, inclusive price boundaries, sorting and clearing were covered by local checks.

3.7 LIMITATIONS

Successful password login, uploads and complete CRUD or review submissions were not verified in the cloud. Live deployment behavior was not independently verified. Automated scans do not certify complete accessibility; manual contrast and assistive-technology checks remain necessary.

Chapter 4

CONCLUSION AND FUTURE WORK

4.1 CONCLUSION

WanderLust integrates server-rendered presentation, database-backed discovery and existing contribution workflows within the original Express and EJS architecture. Visitors can search destinations, compare nightly prices, inspect details and save stays on the current device.

Explicit query validation, stable pagination and actual review summaries make the tested discovery behavior predictable. Reusable templates provide a consistent responsive interface. Fictional demonstration data supplies repeated destinations and varied prices for meaningful search and filter checks.

Recorded unit, HTTP, pagination and browser checks support the tested local behavior. These results are distinct from successful production deployment or complete authentication and upload verification. The application currently provides discovery and listing management.

4.2 FUTURE WORK

- Verify successful authentication, listing, review and upload flows in an isolated test database.
- Add account-based saved stays after approving persistence and access rules.
- Design availability and reservation states, including overlapping-date prevention.
- Introduce payment verification, failure handling and refunds within an agreed booking workflow.
- Measure database performance and extend manual accessibility testing.

Before claiming a live release, verify the existing Render deployment and its critical workflows. A Git push alone does not establish that the deployed service is working.

REFERENCES

[1] WanderLust original project repository.
https://github.com/Namansharma644/WanderLust

[2] Existing WanderLust listings address.
https://wanderlusts-kjty.onrender.com/listings

[3] Node.js documentation.
https://nodejs.org/docs/latest-v22.x/api/

[4] Express routing documentation.
https://expressjs.com/en/guide/routing.html

[5] MongoDB Atlas documentation.
https://www.mongodb.com/docs/atlas/

[6] Mongoose schema documentation.
https://mongoosejs.com/docs/guide.html

[7] EJS template engine documentation.
https://ejs.co/

[8] Bootstrap documentation.
https://getbootstrap.com/docs/5.3/

[9] Passport authentication documentation.
https://www.passportjs.org/

[10] Cloudinary documentation.
https://cloudinary.com/documentation/

[11] Joi validation documentation.
https://joi.dev/api/

[12] Institutional report format supplied by the students.
Mini Project report third year Format.pdf